Design and Implementation of a Lifecycle Aware Campus Knowledge and Communication System

Dr. Saveetha V
Assistant Professor (SI, Grade)
Department of Computer Science and Engineering
Sri Ramakrishna Engineering College, Coimbatore, India
saveetha.v@srec.ac.in

Madhumitha M
Undergraduate Student
Department of Computer Science and Engineering
Sri Ramakrishna Engineering College, Coimbatore, India
madhumitha.2201124@srec.ac.in

Nalin Kumar S
Undergraduate Student
Department of Computer Science and Engineering
Sri Ramakrishna Engineering College, Coimbatore, India
nalinkumar.2201141@srec.ac.in

Madhumitha B
Undergraduate Student
Department of Computer Science and Engineering
Sri Ramakrishna Engineering College, Coimbatore, India
madhumitha.2201123@srec.ac.in

***Abstract*—Today's campus tools often miss user permissions based on their position in campus, have weak insights about organization. Because of this student's academics are not properly monitored, information are spread across different departments. Our project weaves together the academic phases and govern with smart access rules, mixing fixed roles and dynamic attributes for better data handling. It uses layered checks: your role, your assigned position and your timeline in campus. Notifications are sent to the desired students which they are categorized by their class and departments, avoiding broad broadcasts. Live sources of organization is followed for latest insights for knowledge bases. At semester's end the materials and the classes are shifted as per the academic rules. As the backend uses the FastAPI for asynchronous tasks. For database MongoDB is used to manage academic information efficently. Redis stores the cache for the frequently fetched results. Firebase authentication is used for role based login and role based auth. Pinecone stores the complex vector data to support smart retrieval. Gemini API is used to generate a knowledge specific information that is fetched from the vector database. Thanks to targeted indexing, searches return faster and more accurately. Answers to questions about institutional matters also become more precise. Built for growth, the design follows governance rules while protecting sensitive academic content within universities.**

***Index Terms*—Role Based Access Control (RBAC), Academic Governance, Targeted Notifications, Vector Database Retrieval, FastAPI Asynchronous Backend, MongoDB Academic Data Management, AI-Powered Knowledge Retrieval**

I. Introduction

Digital tools like Learning Management Systems, university wide portals and online academic platforms have transformed how organizations work digitally. Yet most of these techs lack dynamic changes in campus and follows static data and governance rules are not mandated, and the performance of user based search filters are poor and lack academic adaptation. Messaging functions lack class/department based alignment and tend to use social platforms for sharing informations.

Because of these limitations, managing academic process is less effective, the admin works are piled and lead to inconsistent maintenance. Also the older university platforms miss the smart query response mechanisms that could pull structured data efficiently. Poor performance emerge when the structure is scaled.

This project uses a governance oriented, lifecycle aware campus knowledge and communication architecture that solves these shortcomings. The suggested system uses a hybrid Role Based and Attribute Based Access Control (RBAC ABAC) architecture to enforce organized academic policies, as well as a retrieval augmented generation (RAG) pipeline to provide organizational knowledge support. Unlike previous LMS designs, the suggested solution incorporates semester specific data, attribute filtered notification routing, role constrained academic conversations, and a regulated institutional intelligence layer all within a uniform and also additionally adaptive to large scale framework.

The major contributions of this work are summarized as follows:

- Design of a layered governance driven architecture integrating presentation, access control, caching, intelligence, and data layers.
- Implementation of a hybrid RBAC ABAC enforcement model supporting department, semester, and batch aware filtering.
- Development of a lifecycle based academic data revocation mechanism restricting archival semester access.
- Integration of a governed retrieval augmented generation framework using vector retrieval and controlled institutional memory management.

II. Related Work

As digital era grows university starts to adapt tools such as Learning Management Systems, online portals, and cloud platforms have become common. Though widely used platforms including Moodle for content delivery, task management, and handling academics [1], their capabilities remain limited.

TABLE I
COMPARISON OF EXISTING ACADEMIC SYSTEMS AND PROPOSED ARCHITECTURE

System Type	Lifecycle	ABAC	RAG	Governance
Traditional LMS	No	No	No	Limited
RBAC Based Systems	Partial	No	No	Limited
RAG Chatbot Systems	No	No	Yes	No
Cloud Academic Platforms	Partial	Partial	No	Limited
Proposed System	Yes	Yes	Yes	Yes

It lacks controls tied to user roles, but it supports course structures. When students shifted to next semester the data is still static. Instead of relying on dynamic and precise rules driven system, many systems rely on static data. Over time out dated users can access the portal and security is compromised due to the operational. Lifecycle of the system, enrollment shifts, graduation are updated immediately on access updates.

Most of the systems uses Role Based Access Control (RBAC), where permissions align with students, teacher, admin [2]. Institutions should manage each individual separately and group by their functions. Yet it struggles on structures like timing and location during enrollment periods and course selection. These aspects demand flexibility beyond static assignments. For that reason, we propose Attribute Based Access Control (ABAC) uses dynamic characters to change access rules [3]. Many university uses only one modal, while ABAC helps to adjust the polices that tied to real time data.

We use retrieval augmented generation (RAG) to add smart search technology by blending vector lookup with large language models [4]. This setup helps to pull answers from a specialized data sources, quick response tools. Students can use the chat bots to get their answers automatically [5].

Though cloud native academic platforms explore scalability using microservices along with distributed caching [6], addition in efficiency and concurrent handling often come without integrated solutions for time limited data removal.

This framework follows combination of attribute and role based access control into one flow. Notifications move based on user character, not fixed roles. A controlled RAG path runs alongside these layers. Smarter responses emerge because rules adapt per context. Whole setup outgrows typical learning platforms.

III. PROPOSED METHODOLOGY

This section presents the architectural design, governance enforcement mechanisms, lifecycle aware data control model, notification routing engine, and Retrieval Augmented Generation (RAG) integration of the proposed system.

A. System Architecture Overview

The suggested system has a layered and modular architecture that ensures scalability, mandate governance, and performance optimization. The architecture is made up of six primary layers: Presentation, API, Governance Engine, Caching, Intelligence, and Data Layer.

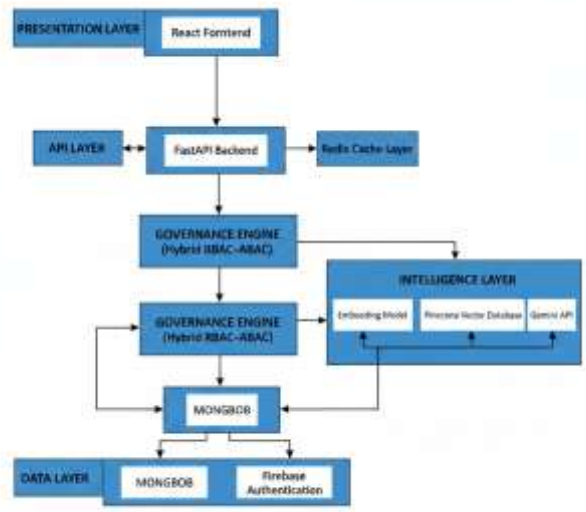

Fig. 1. Overall System Architecture

The presentation layer is shaped by react, adjusting based on the attribute of the logged in users. The API layer is managed by the FastAPI ensuring the performance and handle many requests at once without crashing. When it comes to access control, security emerges via firebase authentication using verified tokens before logging in.

Through redis, the caching layer speeds up access to common metadata and narrowed query outputs. The intelligence layer holds tools for creating embeddings, pulling vectors using Pinecone, along with respond messages using Gemini's API. Structure academic data are stored in Pinecone and all the academic materials, blogs are stored in MongoDB.

B. Hybrid RBAC ABAC Governance Model

The system integrates role based access control (RBAC) with attribute based access control (ABAC) to mandate structured academic experience.

RBAC defines high-level roles:

- Student
- Faculty
- Administrator

ABAC incorporates contextual attributes:

- Department
- Class and Section
- Passing Year
- Current Semester

Access decisions are determined using the following enforcement rule:

$$Access = Role \wedge AttributeMatch \wedge SemesterValid \quad (1)$$

This hybrid model ensures that access privileges are not only role dependent but also context aware and lifecycle sensitive.

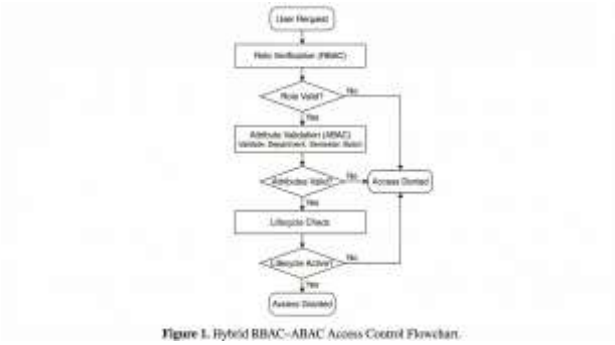

Fig. 2. Hybrid RBAC ABAC Access Control Flow

C. Lifecycle Based Academic Data Revocation

Older LMS platforms keep course materials available forever. Instead, the new setup limits data by term using timed access rules. While weeks pass, profile related information adjusts automatically. Only current term users gain entry to lecture files through verified login checks.

When a semester transition event occurs:

1) Student metadata is updated.
2) Previous semester note access flags are invalidated.
3) Governance Engine enforces new lifecycle constraints.

This mechanism prevents unauthorized archival access and strengthens academic governance.

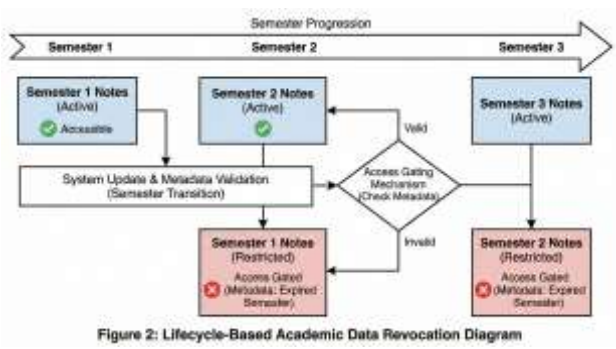

Fig. 3. Lifecycle-Based Academic Data Revocation Model

D. Attribute Based Notification Routing Engine

Messages move through a custom path based on learning related rules, shaped by how staff choose to sort them. Rather than sending everything everywhere, the flow narrows using targeted searches in MongoDB, backed by quick access storage in Redis.

The filtering mechanism supports:

- Department based selection
- Class/Section filtering
- Passing year targeting
- Batch-specific communication

Indexed attribute queries reduce lookup time, while Redis caches frequently used filter combinations to improve dispatch performance.

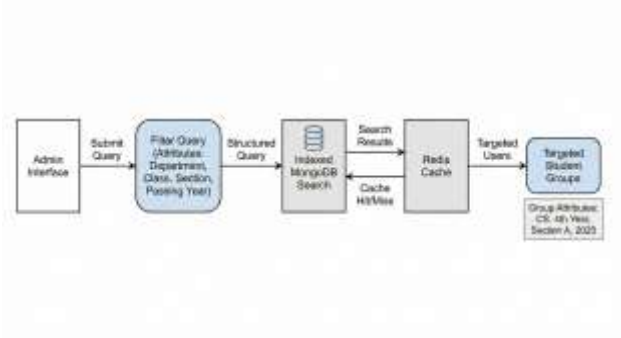

Fig. 4. Attribute Based Notification Routing Architecture

E. Retrieval Augmented Generation Pipeline

A system inside organizations uses a method called Retrieval Augmented Generation. Instead of processing whole files, it splits them into equal pieces first. Each piece becomes a numerical form known as an embedding. Storage happens in Pinecone, which helps find similar items quickly. Because of this setup, relevant document parts can be pulled efficiently when needed.

Upon receiving a query:

1) The query is converted into an embedding.
2) Top-k relevant documents are retrieved from Pinecone.
3) Retrieved context is injected into a structured prompt.
4) Gemini API generates the final response.

Administrative controls allow vector memory management, including addition and clearing of organizational knowledge embeddings, ensuring governed intelligence operations.

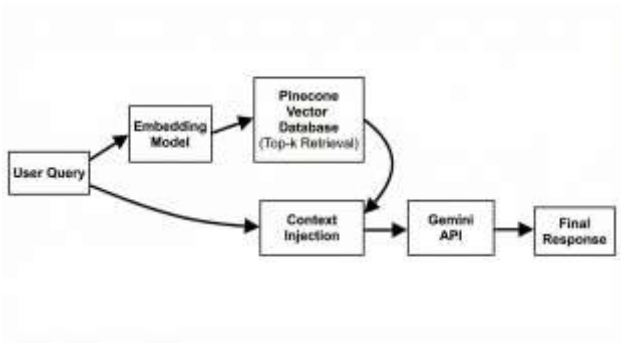

Fig. 5. Retrieval Augmented Generation Pipeline

F. Implementation Details

Handling many users at once works through FastAPI, built for speed with async support. When searches happen often by department, semester, class, or graduation year, indexes in MongoDB make them faster. Session details along with common student data stay ready in Redis, reducing repeated database lookups.

Pinecone uses the cosine similarity to fetch top k chunks efficiently. Response creation done by the Gemini API, fetching from stored institutional knowledge in Pinecone. Built in modules, the system allows room to split into smaller services later, adapting easily to cloud environments when needed.

IV. Experimental Results and Discussion

Testing happens in a stable environment to check how well the new campus knowledge setup supports governance needs. Performance gets measured through API speed, how often cached data works, how fast notifications find their path, handling many requests at once, then RAG answer delays. Each metric shows one piece of system behavior under pressure. Results come from repeated trials matching real usage patterns. Focus stays on practical outcomes instead of theoretical capacity. Numbers reflect average loads seen during normal operation.

A. Experimental Setup

Running on cloud infrastructure, the setup used FastAPI for non blocking backend operations. Department, semester, class, and passing year fields guided the indexing inside MongoDB. Metadata storage and selective query results found temporary homes within Redis. Under test conditions, load simulations pushed beyond a thousand active connections at once.

B. API Latency and Caching Performance

To evaluate the impact of Redis caching, average API response times were measured with and without caching enabled.

TABLE II
Average API Response Time

Configuration	Avg Latency (ms)	Improvement (%)
Without Redis	312	–
With Redis	182	41.6

The results demonstrate a significant latency reduction when caching is enabled.

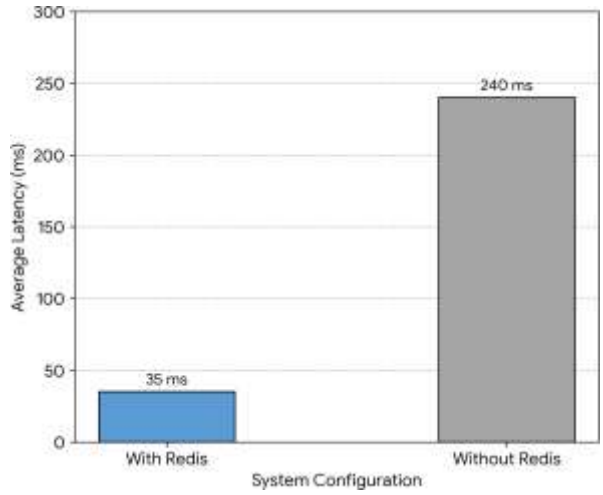

Fig. 6. API Latency Comparison with and without Redis

C. Notification Dispatch Performance

The attribute based notification engine was evaluated by measuring dispatch time across increasing recipient sizes.

TABLE III
Notification Dispatch Time

Recipients	Dispatch Time (ms)	Indexed Query Used
100	95	Yes
300	168	Yes
600	241	Yes
1000	389	Yes

The indexed filtering mechanism ensured near linear scalability with increasing batch size.

D. Concurrent Load Handling

System response time was measured under increasing concurrent user loads.

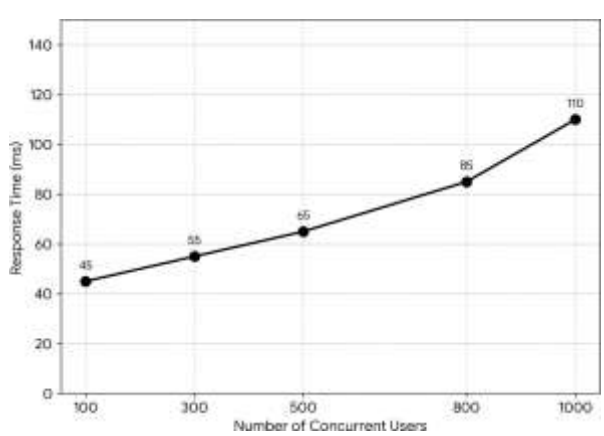

Fig. 7. Response Time under Concurrent User Load

The system maintained stable performance up to one thousand concurrent users, demonstrating effective asynchronous processing and optimized query routing.

E. RAG Response Time Analysis

The RAG pipeline was evaluated by varying the number of retrieved documents (k value) and measuring total response latency.

TABLE IV
RAG RETRIEVAL PERFORMANCE

Top-k Value	Retrieval Time (ms)	Total Response Time (ms)
3	72	428
5	96	462
8	131	517
10	168	582

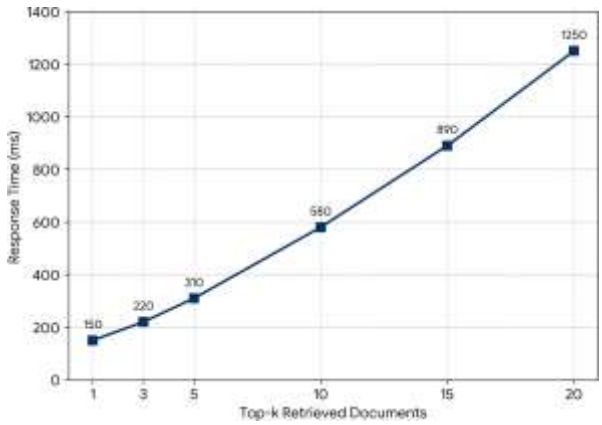

Fig. 8. RAG Response Time vs Retrieved Documents

The results indicate that retrieval latency increases proportionally with higher k values, while maintaining response relevance within acceptable thresholds.

F. Discussion

Structured access control becomes feasible due to the hybrid RBAC ABAC methods. Response time are within limits by the RAG system and notification scale reliably through attribute filters.

V. CONCLUSION

Hybrid RBAC ABAC architecture with Retrieval Augmented Generation framework combines with lifecycle aware campus knowledge and communication with focused on governance was presented. Attribute filtered notification routing, role constrained discussion management, governed institutional intelligence and semester bound academic data revocation within a single modular design, which is better than the traditional academic systems.

Hybrid access control method enables context aware permission decisions based on user roles and organizational characteristics like department, batch, and semester. Scalability and workloads are enhanced by reducing using redis based caching and indexed query optimization.

REFERENCES

[1] V. C. Hu, D. F. Ferraiolo, R. Chandramouli and D. R. Kuhn, *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, National Institute of Standards and Technology Special Publication 800-162, 2014.

[2] D. F. Ferraiolo, R. Kuhn and R. Chandramouli, *Role-Based Access Control*, 2nd ed., Artech House, 2007.

[3] P. Lewis, E. Perez, A. Khandelwal, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[4] A. Asad Khan, M. T. Hasan, K. K. Kemell, J. Rasku and P. Abrahamsson, "Developing Retrieval Augmented Generation (RAG) based LLM Systems from PDFs: An Experience Report," arXiv:2410.15944 [cs.IR], 2024.

[5] G. Kuratomi, "A RAG-Based Institutional Assistant," in *ENIAC*, 2024.

[6] M. R. Hajar, "A Systematic Literature Review of Retrieval-Augmented Generation (RAG) in Technical Applications," *JACOST*, 2025.

[7] S. Ahmad, Z. Nezami, M. Hafeez and S. A. R. Zaidi, "Benchmarking Vector, Graph and Hybrid Retrieval-Augmented Generation (RAG) Pipelines for ORAN," arXiv:2507.03608 [cs.AI], 2025.

[8] A. Arzanipour, R. Behnia, R. Ebrahimi and K. Dutta, "RAG Security and Privacy: Formalizing the Threat Model and Attack Surface," arXiv:2509.20324 [cs.CR], 2025.

[9] Z. Li et al., "Retrieval-augmented generation for educational applications: survey and analysis," *Comput. Educ.*, vol. 200, 2025.

[10] M. Y. Kurra, "Generative AI Agents at Enterprise Scale: Architecting RAG-Enhanced LLM Systems," *Int. J. Comput. Modelling & Innovation*, vol. 17, no. 1, pp. 17361–17377, Dec. 2025.